

PLATFORMS AND INTEGRATIONS

Use Claude Code with Chrome (beta)

Connect Claude Code to your Chrome browser to test web apps, debug with console logs, automate form filling, and extract data from web pages.

Claude Code integrates with the [Claude in Chrome browser extension](#) to give you browser automation capabilities from the CLI or the [VS Code extension](#). Build your code, then test and debug in the browser without switching contexts.

Claude opens new tabs for browser tasks and shares your browser's login state, so it can access any site you're already signed into. Browser actions run in a visible Chrome window in real time. When Claude encounters a login page or CAPTCHA, it pauses and asks you to handle it manually.

ⓘ Chrome integration is in beta and currently works with Google Chrome and Microsoft Edge. It is not yet supported on Brave, Arc, or other Chromium-based browsers. WSL (Windows Subsystem for Linux) is also not supported.

Capabilities

With Chrome connected, you can chain browser actions with coding tasks in a single workflow:

- **Live debugging:** read console errors and DOM state directly, then fix the code that caused them
- **Design verification:** build a UI from a Figma mock, then open it in the browser to verify it matches
- **Web app testing:** test form validation, check for visual regressions, or verify user flows
- **Authenticated web apps:** interact with Google Docs, Gmail, Notion, or any app you're logged into without API connectors
- **Data extraction:** pull structured information from web pages and save it locally
- **Task automation:** automate repetitive browser tasks like data entry, form filling, or multi-site workflows
- **Session recording:** record browser interactions as GIFs to document or share what happened

Prerequisites

Before using Claude Code with Chrome, you need:

- **Google Chrome** or **Microsoft Edge** browser
- **Claude in Chrome extension** version 1.0.36 or higher, available in the Chrome Web Store for both browsers
- **Claude Code** version 2.0.73 or higher
- A direct Anthropic plan (Pro, Max, Team, or Enterprise)

❗ Chrome integration is not available through third-party providers like Amazon Bedrock, Google Cloud Vertex AI, or Microsoft Foundry. If you access Claude exclusively through a third-party provider, you need a separate claude.ai account to use this feature.

Get started in the CLI

1 Launch Claude Code with Chrome

Start Claude Code with the `--chrome` flag:

```
claude --chrome
```

You can also enable Chrome from within an existing session by running `/chrome`.

2 Ask Claude to use the browser

This example navigates to a page, interacts with it, and reports what it finds, all from your terminal or editor:

```
Go to code.claude.com/docs, click on the search box,  
type "hooks", and tell me what results appear
```

Run `/chrome` at any time to check the connection status, manage permissions, reconnect the extension, or choose which connected browser to use. If more than one browser is connected when

a browser action starts, Claude prompts you to pick one.

For VS Code, see [browser automation in VS Code](#).

Enable Chrome by default

To avoid passing `--chrome` each session, run `/chrome` and select “Enabled by default”.

In the [VS Code extension](#), Chrome is available whenever the Chrome extension is installed. No additional flag is needed.

❗ Enabling Chrome by default in the CLI increases context usage since browser tools are always loaded. If you notice increased context consumption, disable this setting and use `--chrome` only when needed.

Manage site permissions

Site-level permissions are inherited from the Chrome extension. Manage permissions in the Chrome extension settings to control which sites Claude can browse, click, and type on.

Example workflows

These examples show common ways to combine browser actions with coding tasks. Run `/mcp` and select `claude-in-chrome` to see the full list of available browser tools.

Test a local web application

When developing a web app, ask Claude to verify your changes work correctly:

```
I just updated the login form validation. Can you open
localhost:3000,
try submitting the form with invalid data, and check if the error
messages appear correctly?
```

Claude navigates to your local server, interacts with the form, and reports what it observes.

Debug with console logs

Claude can read console output to help diagnose problems. Tell Claude what patterns to look for rather than asking for all console output, since logs can be verbose:

```
Open the dashboard page and check the console for any errors when
the page loads.
```

Claude reads the console messages and can filter for specific patterns or error types.

Automate form filling

Speed up repetitive data entry tasks:

```
I have a spreadsheet of customer contacts in contacts.csv. For
each row,
go to the CRM at crm.example.com, click "Add Contact", and fill
in the
name, email, and phone fields.
```

Claude reads your local file, navigates the web interface, and enters the data for each record.

Draft content in Google Docs

Use Claude to write directly in your documents without API setup:

```
Draft a project update based on the recent commits and add it to
my
Google Doc at docs.google.com/document/d/abc123
```

Claude opens the document, clicks into the editor, and types the content. This works with any web app you're logged into: Gmail, Notion, Sheets, and more.

Extract data from web pages

Pull structured information from websites:

Go to the product listings page and extract the name, price, and availability for each item. Save the results as a CSV file.

Claude navigates to the page, reads the content, and compiles the data into a structured format.

Run multi-site workflows

Coordinate tasks across multiple websites:

Check my calendar for meetings tomorrow, then for each meeting with an external attendee, look up their company website and add a note about what they do.

Claude works across tabs to gather information and complete the workflow.

Record a demo GIF

Create shareable recordings of browser interactions:

Record a GIF showing how to complete the checkout flow, from adding an item to the cart through to the confirmation page.

Claude records the interaction sequence and saves it as a GIF file.

Troubleshooting

Extension not detected

If Claude Code can't detect the Chrome extension:

1. Verify the Chrome extension is installed and enabled in `chrome://extensions`
2. Verify Claude Code is up to date by running `claude --version`
3. Check that Chrome is running

4. Run `/chrome` and select “Reconnect extension” to re-establish the connection
5. If the issue persists, restart both Claude Code and Chrome

The first time you enable Chrome integration, Claude Code installs a native messaging host configuration file. Chrome reads this file on startup, so if the extension isn’t detected on your first attempt, restart Chrome to pick up the new configuration.

If the connection still fails, verify the host configuration file exists at:

For Chrome:

- **macOS:** `~/Library/Application Support/Google/Chrome/NativeMessagingHosts/com.anthropic.claude_code_browser_extension.json`
- **Linux:** `~/.config/google-chrome/NativeMessagingHosts/com.anthropic.claude_code_browser_extension.json`
- **Windows:** check `HKCU\Software\Google\Chrome\NativeMessagingHosts\` in the Windows Registry

For Edge:

- **macOS:** `~/Library/Application Support/Microsoft Edge/NativeMessagingHosts/com.anthropic.claude_code_browser_extension.json`
- **Linux:** `~/.config/microsoft-edge/NativeMessagingHosts/com.anthropic.claude_code_browser_extension.json`
- **Windows:** check `HKCU\Software\Microsoft\Edge\NativeMessagingHosts\` in the Windows Registry

Browser not responding

If Claude’s browser commands stop working:

1. Check if a modal dialog (alert, confirm, prompt) is blocking the page. JavaScript dialogs block browser events and prevent Claude from receiving commands. Dismiss the dialog manually, then tell Claude to continue.
2. Ask Claude to create a new tab and try again
3. Restart the Chrome extension by disabling and re-enabling it in `chrome://extensions`

Connection drops during long sessions

The Chrome extension's service worker can go idle during extended sessions, which breaks the connection. If browser tools stop working after a period of inactivity, run `/chrome` and select "Reconnect extension".

Windows-specific issues

On Windows, you may encounter:

- **Named pipe conflicts (EADDRINUSE):** if another process is using the same named pipe, restart Claude Code. Close any other Claude Code sessions that might be using Chrome.
- **Native messaging host errors:** if the native messaging host crashes on startup, try reinstalling Claude Code to regenerate the host configuration.

Common error messages

These are the most frequently encountered errors and how to resolve them:

Error	Cause	Fix
"Browser extension is not connected"	Native messaging host cannot reach the extension	Restart Chrome and Claude Code, then run <code>/chrome</code> to reconnect
"Extension not detected"	Chrome extension is not installed or is disabled	Install or enable the extension in <code>chrome://extensions</code>
"No tab available"	Claude tried to act before a tab was ready	Ask Claude to create a new tab and retry
"Receiving end does not exist"	Extension service worker went idle	Run <code>/chrome</code> and select "Reconnect extension"

See also

- **Computer use**: control native macOS apps when a task can't be done in a browser
- **Use Claude Code in VS Code**: browser automation in the VS Code extension
- **CLI reference**: command-line flags including `--chrome`
- **Common workflows**: more ways to use Claude Code
- **Data and privacy**: how Claude Code handles your data

- **Getting started with Claude in Chrome:** full documentation for the Chrome extension, including shortcuts, scheduling, and permissions